

```

import React, { useState, useEffect, useRef } from 'react';
import { Play, Pause, RotateCcw, ZoomIn, ZoomOut } from 'lucide-react';

const RiemannHelixExplorer = () => {
  const canvasRef = useRef(null);
  const [resolution, setResolution] = useState(400);
  const [yMax, setYMax] = useState(60);
  const [threshold, setThreshold] = useState(99);
  const [useVonMangoldt, setUseVonMangoldt] = useState(true);
  const [showPeaks, setShowPeaks] = useState(true);
  const [showZeros, setShowZeros] = useState(true);
  const [computing, setComputing] = useState(false);
  const [helixAnnotations, setHelixAnnotations] = useState(true);

  // Known Riemann zeros
  const riemannZeros = [14.134725, 21.022040, 25.010858, 30.424876, 32.935062,
    37.586178, 40.918719, 43.327073, 48.005150, 49.773832,
    52.970321, 56.446247, 59.347044];

  // Von Mangoldt function approximation
  const vonMangoldt = (n) => {
    if (n < 2) return 0;
    // Check if n is a prime power
    for (let p = 2; p <= Math.sqrt(n); p++) {
      let pk = p;
      while (pk <= n) {
        if (Math.abs(pk - n) < 0.001) return Math.log(p);
        pk *= p;
      }
    }
    // Check if n itself is prime (simple check)
    let isPrime = n > 1;
    for (let i = 2; i <= Math.sqrt(n); i++) {
      if (n % i === 0) {
        isPrime = false;
        break;
      }
    }
    return isPrime ? Math.log(n) : 0;
  };

  // S_314 function with Von Mangoldt weighting
  const S_314 = (z) => {
    const T = Math.abs(z.im);

```

```

    if (T < 1) return 1.0;

    let sum = 0;
    const nMax = Math.floor(T) + 10;
    for (let n = 2; n <= nMax; n++) {
        const lambda = vonMangoldt(n);
        if (lambda > 0) {
            sum += lambda * Math.sin(T * Math.log(n)) / Math.sqrt(n);
        }
    }
    return 2.5 + 0.5 * Math.sin(2 * Math.PI * T / 10) + 0.3 * sum;
};

// Simple log driver (non-von Mangoldt)
const S_logDriver = (z) => {
    const T = Math.abs(z.im);
    if (T < 1) return 1.0;
    return 2.5 + 1.5 * Math.sin(2 * Math.PI * Math.log(T + 1));
};

// Psi function
const Psi = (z, useVM) => {
    const s_mag = useVM ? S_314(z) : S_logDriver(z);
    const magnitude = Math.sqrt(z.re * z.re + z.im * z.im);
    return s_mag * magnitude * Math.sin(Math.PI * z.re);
};

// Compute heatmap
const computeHeatmap = () => {
    setComputing(true);
    setTimeout(() => {
        const canvas = canvasRef.current;
        if (!canvas) return;

        const ctx = canvas.getContext('2d');
        const width = canvas.width;
        const height = canvas.height;

        const imageData = ctx.createImageData(width, height);
        const data = imageData.data;

        let maxVal = 0;
        const values = [];

        // Compute values
        for (let j = 0; j < height; j++) {
            for (let i = 0; i < width; i++) {

```

```

    const x = i / width;
    const y = (height - j) / height * yMax;

    const z = { re: x, im: y };
    const piEnergy = Math.PI * Math.sqrt(x * x + y * y);
    const psiEnergy = Math.abs(Psi(z, useVonMangoldt));
    const diff = Math.abs(piEnergy - psiEnergy);
    const invDiff = 1.0 / (diff + 0.1);

    values.push(invDiff);
    if (invDiff > maxVal) maxVal = invDiff;
  }
}

// Apply colormap
for (let idx = 0; idx < values.length; idx++) {
  const val = values[idx] / maxVal;
  const i = idx % width;
  const j = Math.floor(idx / width);
  const pixelIndex = (j * width + i) * 4;

  // Plasma-like colormap
  const r = Math.floor(255 * Math.pow(val, 0.5));
  const g = Math.floor(255 * Math.pow(val, 2));
  const b = Math.floor(255 * (0.5 + 0.5 * Math.sin(val * Math.PI)));

  data[pixelIndex] = r;
  data[pixelIndex + 1] = g;
  data[pixelIndex + 2] = b;
  data[pixelIndex + 3] = 255;
}

ctx.putImageData(imageData, 0, 0);

// Draw critical line
const criticalX = 0.5 * width;
ctx.strokeStyle = 'orange';
ctx.lineWidth = 2;
ctx.beginPath();
ctx.moveTo(criticalX, 0);
ctx.lineTo(criticalX, height);
ctx.stroke();

// Draw known zeros
if (showZeros) {
  ctx.fillStyle = 'lime';
  riemannZeros.forEach(zero => {

```

```

        const y = height - (zero / yMax) * height;
        if (y >= 0 && y <= height) {
            ctx.beginPath();
            ctx.arc(criticalX, y, 4, 0, 2 * Math.PI);
            ctx.fill();
        }
    });
}

// Draw helix annotations
if (helixAnnotations) {
    ctx.strokeStyle = 'rgba(255, 255, 0, 0.6)';
    ctx.lineWidth = 2;
    ctx.setLineDash([5, 5]);

    // Draw helical guides
    for (let t = 0; t < yMax; t += 0.5) {
        const y = height - (t / yMax) * height;
        const radius = 15;
        const phase = t * 0.8;
        const x1 = criticalX + radius * Math.cos(phase);
        const x2 = criticalX + radius * Math.cos(phase + Math.PI);

        if (y >= 0 && y <= height) {
            ctx.globalAlpha = 0.4;
            ctx.beginPath();
            ctx.arc(x1, y, 3, 0, 2 * Math.PI);
            ctx.fillStyle = 'cyan';
            ctx.fill();

            ctx.beginPath();
            ctx.arc(x2, y, 3, 0, 2 * Math.PI);
            ctx.fillStyle = 'magenta';
            ctx.fill();
            ctx.globalAlpha = 1.0;
        }
    }
    ctx.setLineDash([]);
}

    setComputing(false);
}, 100);
};

useEffect(() => {
    computeHeatmap();
}, [resolution, yMax, threshold, useVonMangoldt, showZeros, helixAnnotations]);

```

```

return (
  <div className="w-full h-full bg-gray-900 text-white p-6 overflow-auto">
    <div className="max-w-7xl mx-auto">
      <h1 className="text-3xl font-bold mb-2">Riemann Zeros Double Helix Explor
      <p className="text-gray-400 mb-6">
        Visualizing the helical structure in Riemann zeta zeros through Von Man
      </p>

      <div className="grid grid-cols-1 lg:grid-cols-3 gap-6">
        {/* Canvas */}
        <div className="lg:col-span-2">
          <div className="bg-gray-800 rounded-lg p-4">
            <canvas
              ref={canvasRef}
              width={600}
              height={800}
              className="w-full border border-gray-700 rounded"
            />
            {computing && (
              <div className="text-center mt-2 text-yellow-400">Computing...</d
            )}
          </div>
        </div>

        {/* Controls */}
        <div className="space-y-4">
          <div className="bg-gray-800 rounded-lg p-4">
            <h3 className="text-lg font-semibold mb-3">Visualization Controls</

            <div className="space-y-4">
              <div>
                <label className="block text-sm mb-2">
                  Y-axis Range: {yMax}
                </label>
                <input
                  type="range"
                  min="20"
                  max="100"
                  value={yMax}
                  onChange={(e) => setYMax(Number(e.target.value))}
                  className="w-full"
                />
              </div>

              <div>
                <label className="block text-sm mb-2">

```

```

        Resolution: {resolution}
    </label>
    <input
        type="range"
        min="200"
        max="600"
        step="50"
        value={resolution}
        onChange={ (e) => setResolution(Number(e.target.value)) }
        className="w-full"
    />
</div>

<div className="flex items-center space-x-2">
    <input
        type="checkbox"
        id="vonMangoldt"
        checked={useVonMangoldt}
        onChange={ (e) => setUseVonMangoldt(e.target.checked) }
        className="w-4 h-4"
    />
    <label htmlFor="vonMangoldt" className="text-sm">
        Use Von Mangoldt Weighting
    </label>
</div>

<div className="flex items-center space-x-2">
    <input
        type="checkbox"
        id="showZeros"
        checked={showZeros}
        onChange={ (e) => setShowZeros(e.target.checked) }
        className="w-4 h-4"
    />
    <label htmlFor="showZeros" className="text-sm">
        Show Known Zeros (green dots)
    </label>
</div>

<div className="flex items-center space-x-2">
    <input
        type="checkbox"
        id="helixAnnotations"
        checked={helixAnnotations}
        onChange={ (e) => setHelixAnnotations(e.target.checked) }
        className="w-4 h-4"
    />

```

```

        <label htmlFor="helixAnnotations" className="text-sm">
            Show Helix Guides (cyan/magenta)
        </label>
    </div>

    <button
        onClick={computeHeatmap}
        disabled={computing}
        className="w-full bg-blue-600 hover:bg-blue-700 disabled:bg-gra
    >
        {computing ? 'Computing...' : 'Recompute'}
    </button>
</div>
</div>

<div className="bg-gray-800 rounded-lg p-4">
    <h3 className="text-lg font-semibold mb-3">Legend</h3>
    <div className="space-y-2 text-sm">
        <div className="flex items-center space-x-2">
            <div className="w-4 h-4 bg-orange-500"></div>
            <span>Critical line (Re = 0.5)</span>
        </div>
        <div className="flex items-center space-x-2">
            <div className="w-4 h-4 bg-lime-500 rounded-full"></div>
            <span>Known Riemann zeros</span>
        </div>
        <div className="flex items-center space-x-2">
            <div className="w-4 h-4 bg-cyan-400 rounded-full"></div>
            <span>Helix strand 1</span>
        </div>
        <div className="flex items-center space-x-2">
            <div className="w-4 h-4 bg-magenta-400 rounded-full"></div>
            <span>Helix strand 2</span>
        </div>
        <div className="mt-3 pt-3 border-t border-gray-700">
            <p className="text-xs text-gray-400">
                Bright regions show where  $\pi|z| \approx |\Psi(z)|$ . The Von Mangoldt wei
            </p>
        </div>
    </div>
</div>

<div className="bg-gray-800 rounded-lg p-4">
    <h3 className="text-lg font-semibold mb-3">What to Look For</h3>
    <div className="space-y-2 text-sm text-gray-300">
        <p>• Look at the bright spots along the critical line</p>
        <p>• Notice how they form two intertwined strands</p>
    </div>
</div>

```

```
        <p>• The cyan/magenta guides trace potential helix paths</p>
        <p>• Toggle Von Mangoldt weighting to see if the pattern changes<
        <p>• Known zeros (green) align with the helix structure</p>
    </div>
</div>
</div>
</div>
</div>
    );
};

export default RiemannHelixExplorer;
```